C04 Input Manager

CRE132 worksheet

What you'll make

Your first script: **SimplePlayerMovement**, which lets you walk the player around with the arrow keys or WASD, run while holding Left Shift, and press R to jump back to the start.

On the way you'll learn what `Start` and `Update` are, how to read keys and axes with Unity's **Input Manager**, why movement is multiplied by `Time.deltaTime`, and how `[SerializeField]` puts a value in the Inspector so you can tune it without touching the code.



Figure 1: The finished scene: the player in the middle of a grass field with two trees, a bush and a rock.

Before you start

1. Unzip `CRE132_C04_InputManager_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_C04_InputManager_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Core/C04_InputManager`.
3. Press **Play**. Nothing moves yet: the player has no script. Press Play again to stop.

The scene has a **Grid** with a grass tilemap, some **Scenery** (two trees, a bush and a rock) and the **Player**.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Your first script: Start and Update

In the Project window, make a folder `Assets/_game/Scripts`, and inside it a folder `C04_InputManager`. Create a script in it called `SimplePlayerMovement` and double-click it to open it in the code editor.

Unity has written a starting script for you. It has two empty methods (a *method* is a named block of code):

- **`Start`** runs once, when the object first appears.
- **`Update`** runs once every frame, many times a second. Anything that has to keep happening (reading keys, moving) goes here.

Try it: inside the `{ }` of `Start`, type `Debug.Log("Start");` and inside `Update`, type `Debug.Log("Update");`. Save (Ctrl+S) and go back to Unity.

Add it: select the **Player** and drag the script from the Project window onto it (or click **Add Component** and type `SimplePlayerMovement`).

Test it: open the **Console** window (**Window > General > Console**) and press Play. “Start” appears once; “Update” appears over and over, once per frame. Stop, and delete the two `Debug.Log` lines again.

2. The Input Manager

Unity’s old input system is called the **Input Manager**. Open **Edit > Project Settings > Input Manager** and click the arrow next to **Axes**, then open **Horizontal**.

- **Negative Button** `left` and **Positive Button** `right` are the arrow keys. **Alt Negative Button** `a` and **Alt Positive Button** `d` add A and D.
- An *axis* is a number from −1 to 1: −1 when left is held, 1 when right is held, 0 when neither is.
- **Gravity** and **Sensitivity** make the number slide towards 1 and back to 0 rather than jumping. That’s why the movement you’re about to write eases in and out.

Vertical is the same with up/down and W/S. Close the window without changing anything.

3. Walking, running and resetting: SimplePlayerMovement

Open `SimplePlayerMovement` again and replace everything with:

```
// Moves the player with the arrow keys or WASD, using the old Input Manager.
// It changes transform.position directly, so there is no physics yet (that comes in
// C05).
using UnityEngine;

public class SimplePlayerMovement : MonoBehaviour
{
    [SerializeField] private float walkSpeed = 4f;
    [SerializeField] private float runSpeed = 8f;

    // Remembered at the start so the R key can put the player back.
    private Vector3 startPosition;

    private void Start()
    {
        startPosition = transform.position;
    }

    private void Update()
    {
        // GetAxis gives a number from -1 to 1. "Horizontal" and "Vertical" are set up
```

```
        // in Edit > Project Settings > Input Manager (arrow keys and WASD by default).
        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");
        Vector3 direction = new Vector3(horizontal, vertical, 0f);

        // GetKey is true every frame the key is held down.
        float speed = walkSpeed;
        if (Input.GetKey(KeyCode.LeftShift))
        {
            speed = runSpeed;
        }

        // Multiplying by Time.deltaTime turns "units per frame" into "units per
        // second", so the player moves at the same speed on fast and slow computers.
        transform.position = transform.position + direction * speed * Time.deltaTime;

        // GetKeyDown is true only on the one frame the key goes down.
        if (Input.GetKeyDown(KeyCode.R))
        {
            transform.position = startPosition;
            Debug.Log("Player reset to the start position");
        }
    }
}
```

What's going on:

- **Variables** hold values: `float` is a decimal number (`4f` means 4.0), `Vector3` is a position or direction with x, y and z. Lines starting `//` are comments, for people only; Unity ignores them.
- **`Input.GetAxis("Horizontal")`** gives the −1 to 1 number from the Input Manager. The two axes make a `direction`: (1, 0) is right, (0, 1) is up.
- **`Time.deltaTime`** is the time since the last frame, in seconds. A fast computer runs more frames a second, so without it the player would move faster on a fast computer. With it, `speed` means *units per second*.
- **`GetKey` and `GetKeyDown`:** `GetKey` is true on every frame the key is held (running), `GetKeyDown` only on the one frame it goes down (reset once, not every frame you hold R). There is also `GetKeyUp`, for the frame it's released.
- **`[SerializeField] private`** shows `walkSpeed` and `runSpeed` in the Inspector. `startPosition` has no `[SerializeField]`, so it stays hidden.

Save and go back to Unity. If the Console shows red errors, compare your code with the listing above, line by line: a missing `;` or `}` is the usual cause.

Add it: the script is already on the **Player**. Check that **Walk Speed** is **4** and **Run Speed** is **8** in the Inspector.

Test it: press Play and walk around with the arrows or WASD. Hold **Left Shift** to run. Walk away and press **R**: the player jumps back and the Console says "Player reset to the start position".

4. Experiments

Try these, one at a time, putting the code back afterwards:

1. **Without `Time.deltaTime`:** delete `* Time.deltaTime` from the movement line and play. The player shoots off the screen, because it now moves 4 units *every frame*. Put it back.
2. **Tuning in Play mode:** while playing, change **Walk Speed** in the Inspector. The player speeds up straight away. Stop playing: the value goes back. **Changes made in Play mode are lost when you stop**, so write down a value you like and set it again afterwards.

3. **Walking through trees:** walk into a tree. You go straight through it. Moving `transform` ignores physics; C05 fixes that with a Rigidbody 2D.

Check your work

- ☐ Arrows and WASD move the player in all four directions, easing in and out.
- ☐ Holding Left Shift runs at twice the speed.
- ☐ R puts the player back at the start and prints one message per press.
- ☐ Walk Speed and Run Speed show in the Inspector.
- ☐ Compare with `CRE132_C04_InputManager_Finished` if something behaves differently.

Key points

- **`Update` runs once per frame.** Frame rates differ, so multiply movement by `Time.deltaTime`.
- **`GetAxis`** is a smooth number from −1 to 1. **`GetKey`** is held, **`GetKeyDown`** is the one frame it was pressed, **`GetKeyUp`** the frame it was released.
- **Moving `transform` ignores physics**: the player walks straight through trees. C05 fixes that.
- This is the **old Input Manager**. C11 rewrites the same idea with the newer Input System.

If something goes wrong

- **"The referenced script on this Behaviour is missing" or "no MonoBehaviour scripts in the file":** the file name and the class name don't match. Both must be `SimplePlayerMovement`.
- **"; expected" or "} expected" in the Console:** a missing semicolon or bracket. Double-click the error to jump to the line.
- **"The name 'Input' does not exist":** check `using UnityEngine;` is the first line of code and spelt exactly.
- **Nothing moves:** the script isn't on the Player, or Walk Speed is 0 in the Inspector.
- **The player shoots off the screen:** `* Time.deltaTime` is missing from the movement line.

Try it

1. Walk diagonally. Is it faster than walking straight? Why? (Hint: how long is the arrow from (0, 0) to (1, 1)? T01 fixes this with `normalized`.)
2. Swap `GetAxis` for `GetAxisRaw` (both lines). It jumps straight to −1, 0 or 1. Which feels better for a pixel-art game?
3. Add a key that makes the player walk at half speed.